DAYALBAGH EDUCATIONAL INSTITUTE

Faculty of Engineering

Department of Mechanical Engineering

PROJECT REPORT

DESIGN ENGINEERING AND THEME DEVELOPMENT

Course Number: EGC 581 / EGC 681

Project Details

Project Title	VLSI Design of CORDIC Processor for Trigonometric Calculation
Course / Semester	EGC 581 / EGC 681; Semester V
Academic Session	2026–27
Team Number / Group	Individual project (single-member team)
Project Supervisor	

Student Name	Roll Number	Signature
1. Arsh Satsangi	2400941	

Date of Submission: 29 October 2026

Abstract

Digital systems for signal processing, communication and navigation need sine and cosine constantly, but multiplier-based or large look-up-table solutions cost silicon area and power. This project designs a multiplier-free CORDIC (COordinate Rotation DIgital Computer) processor that computes sine and cosine using only shifts, adds and subtracts. The processor works in rotation mode on 32-bit signed Q2.30 fixed-point data and performs 16 micro-rotations, one per clock, so a result is ready 16 clock cycles after the start pulse. The datapath has three registers, three add/subtract units, two shifters and a 16-entry arctangent table, controlled by a 4-bit step counter. The design is written in synthesizable Verilog and verified in Xilinx Vivado with self-checking testbenches against $sin and $cos, supported by a bit-accurate Python model. The worst absolute error is about 3×10^{-5} (roughly 15 bits). A full-circle ($-\pi$ to $+\pi$) wrapper and a one-result-per-clock pipelined version were also developed. Current status: RTL design, simulation and physical layout views are complete.

Keywords: CORDIC, VLSI, Verilog HDL, fixed-point arithmetic, sine and cosine, FPGA

Contents

1. Introduction and Identification of Need....4
1.1 Background....4
1.2 Identified Need....4
1.3 Existing Solutions....4
2. Problem Formulation and Objectives....4
2.1 Problem Statement....4
2.2 Project Objectives....4
2.3 User Needs and Design Specifications....5
3. Generation and Selection of Design Concepts....5
3.1 Concept Selection Matrix....5
4. Feasibility and Final Design....6
4.1 Feasibility Check....6
4.2 Final Design....6
4.3 Important Calculations and Dimensions....8
4.4 Materials and Components....11
5. Implementation / Fabrication....11
5.1 Implementation Steps....11
5.2 RTL Design (Verilog Source Code)....11
5.3 Testbench....14
5.4 Enhancement: Full-Circle Input (cordic_sincos.v)....16
5.5 Enhancement: Pipelined CORDIC (cordic_pipelined.v)....18
5.6 Physical Layout (3D View)....21
6. Testing, Results and Improvements....23
6.1 Test Plan and Results....23
6.2 Problems and Improvements....25
6.3 Final Outcome....25
7. Estimated Cost....26
8. Conclusion and Future Scope....26
8.1 Conclusion....26
8.2 Future Scope....26
9. Team Contribution....26
References....26
Appendix....27
Appendix A. Python reference model....27
Appendix B. File list....28

List of Figures

Figure 1: Iterative CORDIC datapath....6
Figure 2: The start pulse triggers the step counter; the counter drives the shift amount and table index inside the datapath and finally raises the done flag....7
Figure 3: Operation flowchart....8
Figure 4: (a) Rotation of the vector (x, y) by an angle θ gives (x′, y′)....8
Figure 5: Top view of the placed and routed CORDIC processor layout....21
Figure 6: Tilted 3D view of the same layout, showing the cell rows and the vertical power stripes....22
Figure 7: Top view in the highlighted (green) layer view; the wiring is concentrated in the upper part of the core....22
Figure 8: Tilted 3D view in the highlighted (green) layer view....23

1. Introduction and Identification of Need

1.1 Background

This project designs a small processor that computes the sine and cosine of an angle without a hardware multiplier. It uses the CORDIC (COordinate Rotation DIgital Computer) algorithm, which rotates a vector through a series of fixed, shrinking angles. Each rotation needs only two shifts and three add/subtract operations, so the whole design is cheap in area and power. That makes it a good fit for DSP blocks, navigation, communication (NCO / mixers) and motor-control hardware. The algorithm was introduced by Volder [1] and generalised by Walther [2]; its use on FPGAs is surveyed in [3].

The processor is iterative: one micro-rotation per clock, 16 clocks per result. The control is a plain 4-bit synchronous counter rather than a finite-state machine.

1.2 Identified Need

Addressing the need for efficient computation. Digital systems for signal processing, communication and navigation need sine and cosine constantly, and the demand grows as systems get more complex. Straightforward solutions (multiplier-based polynomial series, or large look-up tables) cost silicon area and power, which is a problem for compact, battery-powered and energy-efficient designs.

1.3 Existing Solutions

Existing solution / method	Limitation observed	Possible improvement
Multiplier-based polynomial series	Costs silicon area and power	CORDIC: shifts and adds only, no multipliers
Large look-up tables	Costs silicon area and power	A 16-entry arctangent table with iterative rotation

2. Problem Formulation and Objectives

2.1 Problem Statement

Theme	What it means for this project
Demand for efficient computation	Trigonometric functions must be produced quickly and repeatedly inside larger digital systems.
Designing a CORDIC processor	Build a CORDIC-based processor that computes sine and cosine with shifts and adds only, instead of multipliers.
Focus on area and power	Keep the hardware small: three registers, three add/subtract units, two shifters and a 16-entry table.
Synthesizability and integration	Write plain synthesizable Verilog with a simple interface (clk, rst_n, start, angle_in, done, cos_out, sin_out) so it drops into an existing design.

2.2 Project Objectives

Objective	How this project meets it
Implement CORDIC in rotation mode	Z is driven to zero while X and Y rotate; X ends as cos and Y as sin.
Use the Q2.30 fixed-point format	32-bit words, 2 integer and 30 fraction bits: high precision with plain adders.

Objective	How this project meets it
Develop a synchronous datapath	X, Y, Z registers updated together on every clock edge with non-blocking assignments.
Control iterations with counter logic	A 4-bit step counter selects the shift amount and the table entry; no FSM is needed.
Verify with a testbench	Self-checking testbench compares against $sin and $cos and prints PASS / FAIL.
Analyze waveforms for accuracy	Watch X, Y, Z converge over 16 clocks in Vivado and compare with the expected hex values.

2.3 User Needs and Design Specifications

Requirement / feature	Must-have or desirable	Target / specification	How it will be checked
Sine and cosine without a multiplier	Must-have	Only shifts, adds and subtracts	Inspection of the Verilog source
Accuracy	Must-have	Error about 3e-5 (testbench tolerance 1e-4)	Self-checking testbench against $sin and $cos
Latency	Must-have	16 clock cycles per result	Simulation (done flag, cycle count)
Input range	Must-have	θ in [−π/2, +π/2], Q2.30	Test angles from −90° to +90°
Simple synthesizable interface	Must-have	clk, rst_n, start, angle_in, done, cos_out, sin_out	Vivado simulation and inspection
Full-circle input	Desirable	−π to +π by quadrant folding (Q3.29 input)	tb_cordic_sincos.v
Higher throughput	Desirable	One result per clock (latency 17 clocks)	tb_cordic_pipelined.v

3. Generation and Selection of Design Concepts

Concept	Sketch / figure no.	Working principle	Main advantage
Concept A: Iterative CORDIC (cordic_core)	Figures 1 to 3	One micro-rotation per clock using one set of X, Y, Z registers, add/subtract units and shifters; 16 clocks per result	Small area and power
Concept B: Pipelined CORDIC (cordic_pipelined)	Section 5.5	Each of the 16 iterations has its own hardware and pipeline register	One result per clock (latency 17 clocks)

3.1 Concept Selection Matrix

Criterion	Concept A: Iterative	Concept B: Pipelined
Meets user need	Sine and cosine with shifts and adds only	Same arithmetic as Concept A
Ease of construction	Three registers, three add/subtract units, two shifters and a 4-bit counter	Each iteration has its own hardware; the shifters become fixed wiring
Cost (area and power)	Small	Area grows about 16 times for the adders and registers
Throughput	One result every 16 clocks	One result per clock (latency 17 clocks)
Safety / ease of use	Simple interface: clk, rst_n, start, angle_in, done, cos_out, sin_out	Streaming interface: valid_in, valid_out

Selected concept and reason: Concept A (the iterative core) is the main design, because the project focuses on keeping the hardware small for area and power. Concept B is kept as an enhancement for higher throughput.

4. Feasibility and Final Design

4.1 Feasibility Check

Aspect	Finding / observation	Action required (if any)
Practical / physical feasibility	Plain synthesizable Verilog with a simple interface (clk, rst_n, start, angle_in, done, cos_out, sin_out); drops into an existing design	None
Technical feasibility	Error about 3e-5 (roughly 15 bits) over the input range ±π/2; testbench tolerance 1e-4	Angles outside ±π/2 are folded first (Section 5.4)
User comfort and safety	Not applicable: a digital hardware block with no user-handled parts	None
Cost and resource availability	Small hardware: three registers, three add/subtract units, two shifters and a 16-entry table; no multiplier	None

4.2 Final Design

Block diagram

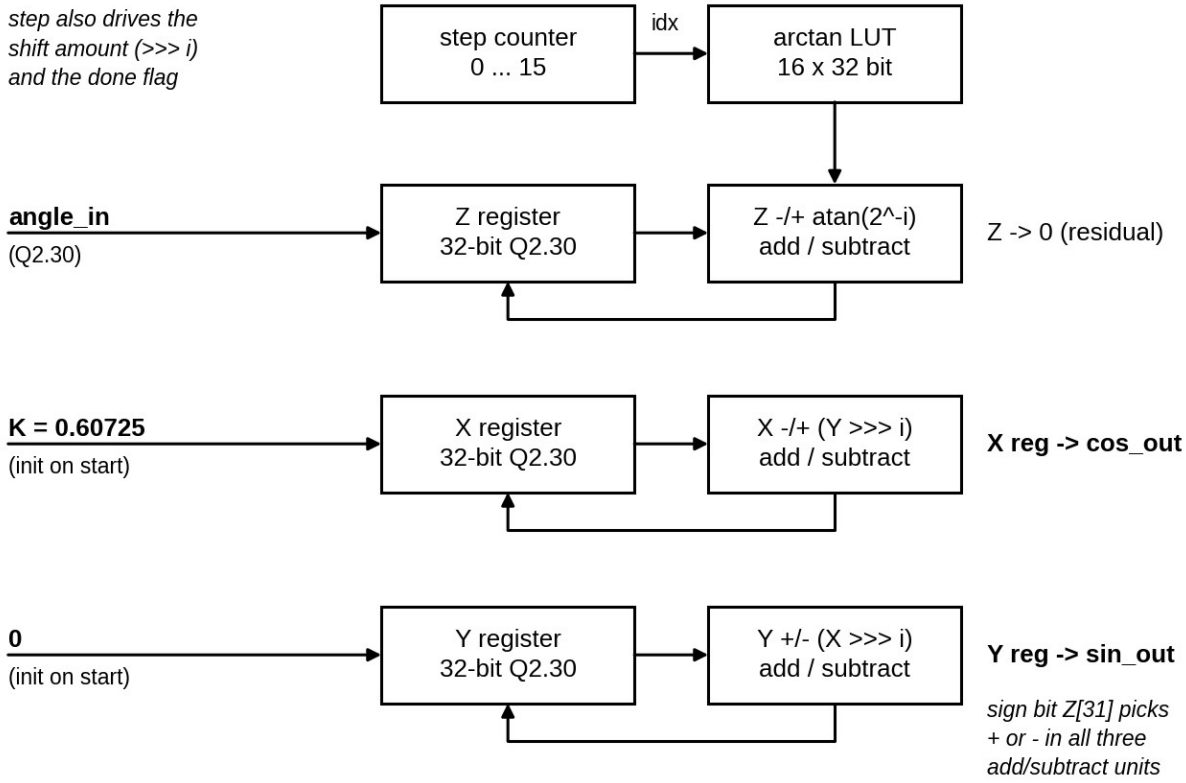


Figure 1: Iterative CORDIC datapath. Each clock the three registers are updated together from their old values.

Operation flow

1. **Idle:** busy = 0. Outputs hold the last result; done stays 1 after a finished run.
2. **Start pulse** (one clock): X ← K, Y ← 0, Z ← angle_in, step ← 0, busy ← 1, done ← 0.

3. **Micro-rotation (16 clocks, step = 0...15):** if Z[31] = 1 (negative): X ← X + (Y>>>step), Y ← Y − (X>>>step), Z ← Z + atan[step]. Otherwise: X ← X − (Y>>>step), Y ← Y + (X>>>step), Z ← Z − atan[step]. Then step ← step + 1.
4. **Finish:** on the clock where step = 15 the registers take their last update, busy ← 0 and done ← 1.
5. **Read:** while done = 1, cos_out = X and sin_out = Y.

Timing

Clock edge	What happens	step value used
N (start = 1)	Registers load K, 0, angle_in	-
N+1	Micro-rotation 0	0
N+2	Micro-rotation 1	1
...	...	...
N+16	Micro-rotation 15, done ← 1	15
after N+16	cos_out, sin_out valid	-

A new start pulse reloads the registers and restarts the run. Non-blocking assignments make all three updates in one clock use the old X, Y and Z values, as in hardware.

System-level view (control and datapath)

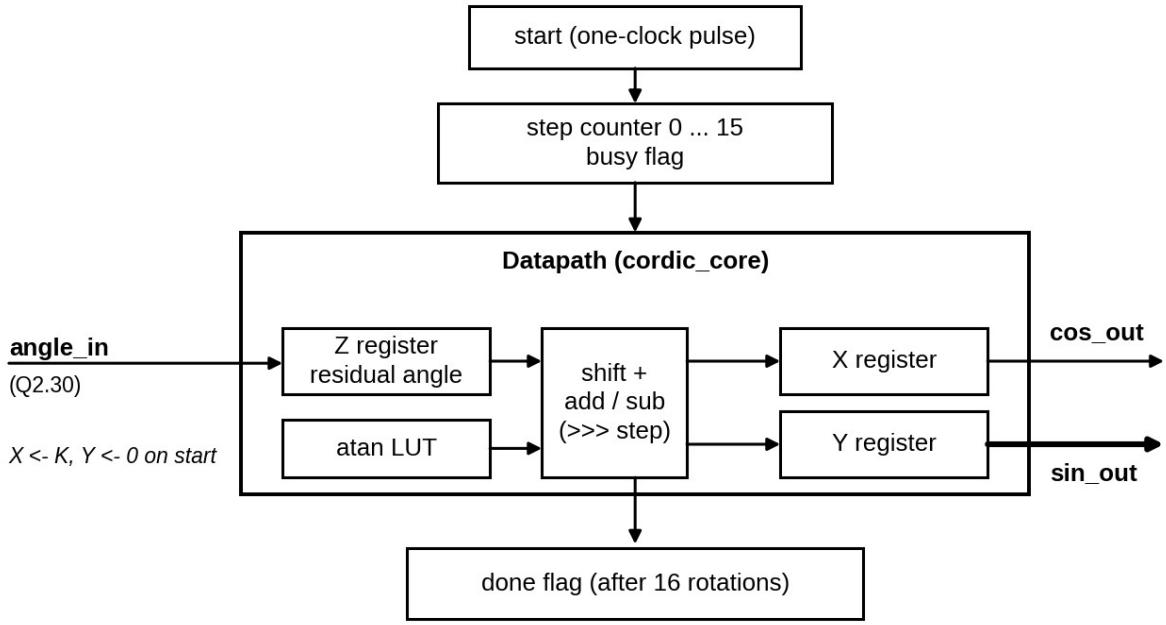


Figure 2: The start pulse triggers the step counter; the counter drives the shift amount and table index inside the datapath and finally raises the done flag.

Flowchart of the operation

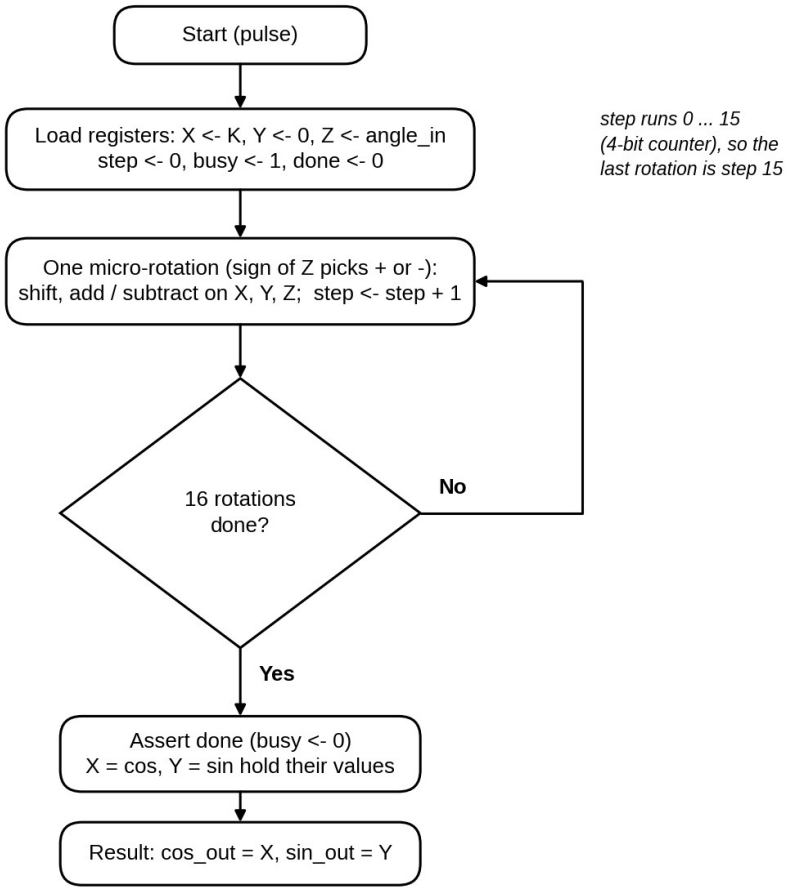


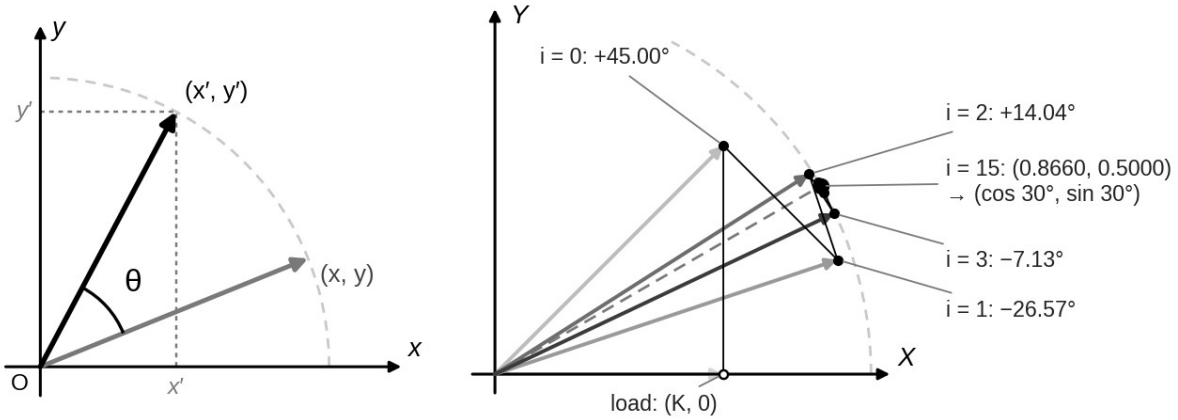
Figure 3: Operation flowchart. The loop repeats once per clock until the sixteenth rotation has been performed.

4.3 Important Calculations and Dimensions

4.3.1 CORDIC

Rotating a vector (x, y) by an angle θ gives (Figure 4):

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$
$$y' = y \cdot \cos\theta + x \cdot \sin\theta$$



(a) Rotating (x, y) by an angle θ (b) CORDIC micro-rotations for θ = 30° (values from Section 6.1)

Figure 4: (a) Rotation of the vector (x, y) by an angle θ gives (x′, y′). (b) CORDIC reaches θ = 30° through 16 shrinking micro-rotations; the tip of the vector spirals in on (cos 30°, sin 30°) while the residual angle Z shrinks to zero.

Factor out cosθ: x' = cosθ·(x − y·tanθ) and y' = cosθ·(y + x·tanθ). CORDIC splits θ into small steps θ_i chosen so that $\tan \theta_i = 2^{-i}$. Then multiplying by $\tan\theta_i$ is just a right shift by i bits. Each step may be applied clockwise or counter-clockwise ($d_i = \pm 1$):

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i}$$
$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i}$$
$$z_{i+1} = z_i - d_i \cdot \operatorname{atan}(2^{-i})$$
$$d_i = +1 \text{ if } z_i \geq 0, \text{ else } -1$$

The cosθ factors are dropped from each step, so each micro-rotation also stretches the vector by $\operatorname{sqrt}(1 + 2^{-2i})$. After 16 steps the total stretch is

$$A_{16} = \Pi \operatorname{sqrt}(1 + 2^{-2i}) = 1.646760258, \quad K = 1 / A_{16} = 0.607252935$$

Starting with $x_0 = K$, $y_0 = 0$, $z_0 = \theta$ cancels the stretch in advance. When z reaches 0 the vector has been rotated by θ, so $x_{16} \approx \cos\theta$ and $y_{16} \approx \sin\theta$.

Convergence range

The largest total rotation available is $\Sigma \operatorname{atan}(2^{-i})$ for i = 0...15 = 1.74326 rad (99.88°). The input must be inside this range, so this design accepts θ in [−π/2, +π/2] (±1.5708 rad). Larger angles are folded into that range first (Section 5.4).

Accuracy

After the last step the leftover angle is at most $\operatorname{atan}(2^{-15}) \approx 3.05\text{e-}5$ rad, which limits the error of sin and cos to about 3e-5 (roughly 15 correct bits). Truncation in the shifts adds only a few units of 2^{-30}. More iterations give more accuracy (the LUT then needs more entries). The measured worst error over a 0.5° sweep of the full range was 3.01e-05.

Core mechanism in four points

Point	Explanation
Iterative vector rotation	The vector (X, Y) is turned toward the target angle through a series of rotations, each a little smaller than the last.
Micro-rotation from the residual angle	At every iteration the sign of the remaining angle Z decides whether the next small rotation goes clockwise or counter-clockwise, so the leftover angle shrinks toward zero.
Shifts replace division and multiplication	Each step multiplies by 2^{-i}, which is just a right shift by i bits. Only shifts, adds and subtracts are needed.
Final outputs	When the residual angle is about zero, X holds cos(θ) and Y holds sin(θ).

4.3.2 Fixed-point format Q2.30

A 32-bit two's-complement word is read as a signed integer N and its real value is $N / 2^{30}$.

Quantity	Value
Bits	1 sign + 1 integer + 30 fraction (written Q2.30)
Range	−2.0 ... +1.999999999
Resolution (1 LSB)	$2^{-30} = 9.31\text{e-}10$
1.0	0x40000000
π/2 = 1.5707963	0x6487ED51 (1686629713)
−1.0	0xC0000000

Converting a real number r to Q2.30: $N = \operatorname{round}(r \times 2^{30})$. Converting back: $r = N / 2^{30}$. Example: 0.5 → 0x20000000.

- The vector length grows from K ≈ 0.607 to 1.0, so X and Y never exceed ±1.0 plus a tiny margin. They always fit in ±2.
- π/2 ≈ 1.5708 fits (π ≈ 3.14 does not), which is why the input range is ±π/2.
- Add and subtract work on the raw integers with no scaling, and arithmetic shift right (>>>) divides a signed value by 2^i while keeping its sign.
- In Verilog the shift only behaves as arithmetic when the operand is declared signed. All data registers and wires in the design are signed [31:0].
- Scaling of angles, constants and outputs are shown in the following Table:

Quantity	**To hardware (real → Q2.30)**	**Example**
Input angle	$N = \text{round}(\theta \times 2^{30})$	30° = 0.5236 rad → 0x2182A470
Arctangent constants	$N = \text{round}(\text{atan}(2^{-i}) \times 2^{30})$	i = 0 → 0x3243F6A9
Gain constant K	$N = \text{round}(0.60725 \times 2^{30})$	0x26DD3B6A
Outputs cos, sin	$\text{real} = N / 2^{30}$	0x376CD741 → 0.866025

All values stay inside ±2, so one common scale factor keeps every computation inside the desired precision range.

4.3.3 Constants: arctan table and gain K

Every constant below is computed as round(value × 2^{30}) and was checked against Python's math library.

Arctangent table (used by cordic_atan_lut.v)

i	**atan(2^{-i}) rad**	**degrees**	**Q2.30 hex**	**Q2.30 decimal**
0	0.785398163	45.00000	0x3243F6A9	843314857
1	0.463647609	26.56505	0x1DAC6705	497837829
2	0.244978663	14.03624	0x0FADBAFD	263043837
3	0.124354995	7.12502	0x07F56EA7	133525159
4	0.062418810	3.57633	0x03FEAB77	67021687
5	0.031239833	1.78991	0x01FFD55C	33543516
6	0.015623729	0.89517	0x00FFFAAB	16775851
7	0.007812341	0.44761	0x007FFF55	8388437
8	0.003906230	0.22381	0x003FFFEB	4194283
9	0.001953123	0.11191	0x001FFFFD	2097149
10	0.000976562	0.05595	0x00100000	1048576
11	0.000488281	0.02798	0x00080000	524288
12	0.000244141	0.01399	0x00040000	262144
13	0.000122070	0.00699	0x00020000	131072
14	0.000061035	0.00350	0x00010000	65536
15	0.000030518	0.00175	0x00008000	32768

Gain compensation constant

Quantity	**Value**
$A_{16} = \Pi \text{ sqrt}(1+2^{-2i})$	1.6467602579
$K = 1 / A_{16}$	0.6072529351
K in Q2.30 (hex)	0x26DD3B6A
K in Q2.30 (decimal)	652032874

4.4 Materials and Components

Parameter	Choice	Reason for Selection
HDL	Verilog (IEEE 1364-2001/2005)	Portable across Vivado xsim and Icarus Verilog
EDA tool	Xilinx Vivado	RTL simulation, waveform viewer, synthesis, utilization and timing reports
Mode	Rotation (z driven to 0)	Input is an angle; outputs are cos and sin
Data format	Q2.30 signed, 32-bit	Range -2 ... +1.999999999, resolution $2^{-30} \approx 9.3e-10$
Iterations	16	Residual angle ≤ $\text{atan}(2^{-15}) \approx 3.05e-5$ rad
Division	Right shifts only	$x / 2^i$ becomes x >>> i (arithmetic shift keeps sign)
Control	4-bit counter + busy/done flags	Simpler than an FSM, one rotation per clock
Reset	Active-low asynchronous (rst_n)	Common FPGA practice

Datapath registers

Register	Reset/start value	Meaning
X	K = 0.607252935 (0x26DD3B6A)	Becomes cos(θ)
Y	0	Becomes sin(θ)
Z	angle_in (Q2.30)	Residual angle, driven toward 0

5. Implementation / Fabrication

5.1 Implementation Steps

Step	Work carried out	Tools / resources used
1	Defined the Q2.30 format and computed the arctangent table and the gain constant K	Python (cordic_model.py)
2	Wrote the RTL: cordic_atan_lut.v and cordic_core.v	Verilog, Xilinx Vivado
3	Wrote the self-checking testbench tb_cordic_core.v and simulated the design	Xilinx Vivado simulator
4	Added the enhancements cordic_sincos.v and cordic_pipelined.v with their testbenches	Verilog, Xilinx Vivado
5	Cross-checked the results against the bit-accurate Python model	Python
6	Produced the physical layout views of the design (Section 5.6)	Layout views

Source Codes Developed:

- cordic_atan_lut.v: 16-entry arctangent table in Q2.30
- cordic_core.v: datapath (X, Y, Z registers, shifters, add/subtract) plus the step counter and done flag
- tb_cordic_core.v: self-checking testbench that compares against $sin and $cos
- cordic_sincos.v (enhancement): wrapper that accepts any angle from −π to +π
- cordic_pipelined.v (enhancement): one-result-per-clock version
- tb_cordic_sincos.v and tb_cordic_pipelined.v: testbenches for the two enhancements

5.2 RTL Design (Verilog Source Code)

All files use `timescale 1ns / 1ps and are written in plain Verilog-2001 so they compile in Vivado and Icarus Verilog.

RTL design overview

- **Implementation in Verilog HDL:** a hardware description language that models the registers, adders and control exactly as they will be built.
- **Modules for datapath and arctangent lookup:** cordic_core holds the datapath and counter; cordic_atan_lut is a separate 16-entry table module instantiated inside it.
- **Use of non-blocking assignments:** every clocked register uses <=, so X, Y and Z all update from their old values in the same clock, as real flip-flops do. Combinational blocks (the table, the folding logic) use always @(*) with blocking assignments.
- **Sequential control driven by a counter:** the 4-bit step counter, busy and done flags replace a state machine.

```
cordic_sincos  (optional wrapper: quadrant folding)
 └── cordic_core  (datapath + step counter + done flag)
      └── cordic_atan_lut  (16 x 32-bit arctangent table)
```

5.2.1 cordic_atan_lut.v

```
// ============================================================================
// cordic_atan_lut.v
// Arctangent look-up table for CORDIC, Q2.30 fixed point.
// entry[i] = round( atan(2^-i) * 2^30 ), i = 0 .. 15
// ============================================================================
`timescale 1ns / 1ps
module cordic_atan_lut (
    input  wire [3:0] idx,              // iteration number 0..15
    output reg  signed [31:0] atan      // atan(2^-idx) in Q2.30
);
    always @(*) begin
        case (idx)
            4'd0 : atan = 32'h3243F6A9; // atan(2^-0) = 0.785398163 rad (45 deg)
            4'd1 : atan = 32'h1DAC6705; // atan(2^-1) = 0.463647609
            4'd2 : atan = 32'h0FADBAFD; // atan(2^-2) = 0.244978663
            4'd3 : atan = 32'h07F56EA7; // atan(2^-3) = 0.124354995
            4'd4 : atan = 32'h03FEAB77; // atan(2^-4) = 0.062418810
            4'd5 : atan = 32'h01FFD55C; // atan(2^-5) = 0.031239833
            4'd6 : atan = 32'h00FFFAAB; // atan(2^-6) = 0.015623729
            4'd7 : atan = 32'h007FFF55; // atan(2^-7) = 0.007812341
            4'd8 : atan = 32'h003FFFEB; // atan(2^-8) = 0.003906230
            4'd9 : atan = 32'h001FFFFD; // atan(2^-9) = 0.001953123
            4'd10: atan = 32'h00100000; // atan(2^-10) = 0.000976562
            4'd11: atan = 32'h00080000; // atan(2^-11) = 0.000488281
            4'd12: atan = 32'h00040000; // atan(2^-12) = 0.000244141
            4'd13: atan = 32'h00020000; // atan(2^-13) = 0.000122070
            4'd14: atan = 32'h00010000; // atan(2^-14) = 0.000061035
            4'd15: atan = 32'h00008000; // atan(2^-15) = 0.000030518
            default: atan = 32'h00000000;
        endcase
    end
endmodule
```

5.2.2 cordic_core.v (top module of the design)

```
// ============================================================================
// cordic_core.v
// Iterative CORDIC processor, ROTATION mode, Q2.30 fixed point.
//
// Input  : angle_in (Q2.30 radians, valid range -pi/2 .. +pi/2)
// Output : cos_out (Q2.30)  <- X register
//          sin_out (Q2.30)  <- Y register
// Latency: 16 clock cycles after the start pulse (1 micro-rotation / clock)
//
// Datapath registers
//   X <- K (CORDIC gain compensation constant = 0.607252935)
//   Y <- 0
//   Z <- angle_in (residual angle)
//
// Each micro-rotation i (0..15):
//   if Z >= 0 : X' = X - (Y >>> i)   Y' = Y + (X >>> i)   Z' = Z - atan(2^-i)
//   else      : X' = X + (Y >>> i)   Y' = Y - (X >>> i)   Z' = Z + atan(2^-i)
//
// Only shifts, adders and subtractors are used - no multipliers.
// ============================================================================
`timescale 1ns / 1ps
```

```
module cordic_core (
    input  wire               clk,
    input  wire               rst_n,     // active-low asynchronous reset
    input  wire               start,     // 1-clock pulse: load angle, begin
    input  wire signed [31:0] angle_in,  // Q2.30 radians
    output reg                done,      // high when outputs are valid
    output wire signed [31:0] cos_out,   // Q2.30
    output wire signed [31:0] sin_out    // Q2.30
);
    // CORDIC gain compensation: K = prod(1/sqrt(1+2^-2i)), i=0..15
    // K = 0.6072529351 -> round(K * 2^30) = 0x26DD3B6A
    localparam signed [31:0] K_INIT = 32'h26DD3B6A;

    // ------------------------------------------------------------------
    // Datapath registers
    // ------------------------------------------------------------------
    reg signed [31:0] x_reg;
    reg signed [31:0] y_reg;
    reg signed [31:0] z_reg;

    // ------------------------------------------------------------------
    // Control: synchronous 4-bit step counter (replaces an FSM)
    // ------------------------------------------------------------------
    reg [3:0] step;   // 0..15
    reg       busy;

    // ------------------------------------------------------------------
    // Arctangent LUT
    // ------------------------------------------------------------------
    wire signed [31:0] atan_val;
    cordic_atan_lut u_lut (
        .idx  (step),
        .atan (atan_val)
    );

    // ------------------------------------------------------------------
    // Shifted operands (arithmetic right shift keeps the sign)
    // ------------------------------------------------------------------
    wire signed [31:0] x_shift = x_reg >>> step;
    wire signed [31:0] y_shift = y_reg >>> step;

    // Direction: z negative -> rotate clockwise (d = -1)
    wire z_neg = z_reg[31];

    // ------------------------------------------------------------------
    // Sequential logic (non-blocking assignments only)
    // ------------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            x_reg <= 32'sd0;
            y_reg <= 32'sd0;
            z_reg <= 32'sd0;
            step  <= 4'd0;
            busy  <= 1'b0;
            done  <= 1'b0;
        end
        else if (start) begin
            // Initialise datapath
            x_reg <= K_INIT;
            y_reg <= 32'sd0;
            z_reg <= angle_in;
            step  <= 4'd0;
            busy  <= 1'b1;
            done  <= 1'b0;
        end
        else if (busy) begin
            if (z_neg) begin
                x_reg <= x_reg + y_shift;
                y_reg <= y_reg - x_shift;
                z_reg <= z_reg + atan_val;
            end
            else begin
                x_reg <= x_reg - y_shift;
                y_reg <= y_reg + x_shift;
                z_reg <= z_reg - atan_val;
            end
            step <= step + 4'd1;
            if (step == 4'd15) begin
                busy <= 1'b0;
                done <= 1'b1;            // stays high until next start
            end
        end
    end

    assign cos_out = x_reg;
    assign sin_out = y_reg;
endmodule
```

Code walk-through

- **K_INIT** is the gain-compensated starting value of X.
- **x_shift / y_shift** are the arithmetic right shifts by the current step number. A variable shift synthesizes into a barrel shifter (a few LUT levels).
- **z_neg = z_reg[31]** is the sign test that decides the direction d.
- **start** has priority over everything, so it also works as a synchronous restart.
- **step == 15** is the last iteration; one clock later the outputs are the final result and done is 1.
- **cos_out / sin_out** are direct connections to X and Y: no extra output register is needed.

5.3 Testbench

5.3.1 tb_cordic_core.v

```
// ============================================================================
// tb_cordic_core.v
// Self-checking testbench for cordic_core (Q2.30).
// Works in Xilinx Vivado Simulator (xsim) and Icarus Verilog (-g2005).
// ============================================================================
`timescale 1ns / 1ps
module tb_cordic_core;

    // ---------------- DUT signals ----------------
    reg                clk;
    reg                rst_n;
    reg                start;
    reg  signed [31:0] angle_in;
    wire               done;
    wire signed [31:0] cos_out;
    wire signed [31:0] sin_out;

    cordic_core dut (
        .clk      (clk),
        .rst_n    (rst_n),
        .start    (start),
        .angle_in (angle_in),
        .done     (done),
        .cos_out  (cos_out),
        .sin_out  (sin_out)
    );

    // ---------------- Clock: 10 ns period (100 MHz) ----------------
    initial clk = 1'b0;
    always #5 clk = ~clk;

    // ---------------- Helpers ----------------
    real SCALE;          // 2^30
    real TOL;            // allowed absolute error
    integer errors;
    integer tests;

    function real q230_to_real;
        input signed [31:0] v;
        begin
            q230_to_real = $itor(v) / 1073741824.0;
        end
    endfunction

    function signed [31:0] real_to_q230;
        input real r;
        begin
            real_to_q230 = $rtoi(r * 1073741824.0);
        end
    endfunction

    function real absr;
        input real r;
        begin
            absr = (r < 0.0) ? -r : r;
        end
    endfunction

    // ---------------- One test: apply angle, wait for done, compare ----------------
    task run_test;
        input real angle_rad;
        real exp_c, exp_s, got_c, got_s, err_c, err_s;
        integer cyc;
        begin
```

```
            tests = tests + 1;
            angle_in = real_to_q230(angle_rad);
            @(negedge clk);
            start = 1'b1;
            @(negedge clk);
            start = 1'b0;

            // wait for done (timeout guard)
            cyc = 0;
            while (!done && cyc < 100) begin
                @(negedge clk);
                cyc = cyc + 1;
            end

            exp_c = $cos(angle_rad);
            exp_s = $sin(angle_rad);
            got_c = q230_to_real(cos_out);
            got_s = q230_to_real(sin_out);
            err_c = absr(got_c - exp_c);
            err_s = absr(got_s - exp_s);

            $display("angle=%9.6f rad (%8.3f deg)  cycles=%0d",
                     angle_rad, angle_rad*57.29577951, cyc);
            $display("  cos: exp=%9.6f got=%9.6f err=%e", exp_c, got_c, err_c);
            $display("  sin: exp=%9.6f got=%9.6f err=%e", exp_s, got_s, err_s);
            $display("  hex: X(cos)=%h  Y(sin)=%h", cos_out, sin_out);

            if (!done || err_c > TOL || err_s > TOL) begin
                $display("  ** FAIL **");
                errors = errors + 1;
            end
            else
                $display("  PASS");
            repeat (2) @(negedge clk);
        end
    endtask

    // ---------------- Stimulus ----------------
    integer k;
    real    a;
    initial begin
        $dumpfile("cordic_tb.vcd");
        $dumpvars(0, tb_cordic_core);
        SCALE = 1073741824.0;
        TOL   = 1.0e-4;            // 16 iterations -> about 2^-15 accuracy
        errors = 0;
        tests  = 0;
        rst_n = 1'b0;
        start = 1'b0;
        angle_in = 32'sd0;
        repeat (4) @(negedge clk);
        rst_n = 1'b1;
        repeat (2) @(negedge clk);

        $display("==== CORDIC rotation-mode test (Q2.30, 16 iterations) ====");

        // Corner and typical angles (radians)
        run_test( 0.0);
        run_test( 0.5235987756);   //  30 deg
        run_test( 0.7853981634);   //  45 deg
        run_test( 1.0471975512);   //  60 deg
        run_test( 1.5707963268);   //  90 deg (pi/2)
        run_test(-0.5235987756);   // -30 deg
        run_test(-0.7853981634);   // -45 deg
        run_test(-1.0471975512);   // -60 deg
        run_test(-1.5707963268);   // -90 deg
        run_test( 0.1);
        run_test(-0.2);
        run_test( 1.2345);

        // Sweep -pi/2 .. +pi/2 in 0.1 rad steps
        for (k = -15; k <= 15; k = k + 1) begin
            a = k * 0.1;
            run_test(a);
        end

        $display("==== Done: %0d tests, %0d errors ====", tests, errors);
        if (errors == 0) $display("ALL TESTS PASSED");
        else             $display("SOME TESTS FAILED");
        $finish;
    end

    // Optional: cycle-by-cycle view of convergence.
    // Sampled on the falling edge, so the values shown are the registers AFTER
    // the rising edge: "rotations done" = 0 is the freshly loaded K, 0, angle.
    reg done_q;
    always @(posedge clk) done_q <= done;
    always @(negedge clk)
        if (dut.busy || (done && !done_q))
```

```
            $display("    rotations done=%2d  X=%f  Y=%f  Z=%f",
                     (done ? 16 : dut.step),
                     q230_to_real(dut.x_reg),
                     q230_to_real(dut.y_reg),
                     q230_to_real(dut.z_reg));
endmodule
```

5.4 Enhancement: Full-Circle Input (cordic_sincos.v)

The core accepts [−π/2, +π/2]. Q2.30 cannot represent π (its limit is ±2), so the wrapper takes the angle in Q3.29 (range ±4) and folds it into the core's range with these identities:

$\theta > \pi/2 : \theta' = \pi - \theta$, $\cos\theta = -\cos\theta'$, $\sin\theta = \sin\theta'$

$\theta < -\pi/2 : \theta' = -\pi - \theta$, $\cos\theta = -\cos\theta'$, $\sin\theta = \sin\theta'$

otherwise: $\theta' = \theta$

Hardware cost: two comparators, one subtractor, a 2-to-1 mux, a one-bit shift (Q3.29 → Q2.30), one flag register and one conditional negate on the cosine output. Latency is still 16 clocks. Inputs outside ±π are not supported. Note that π in Q3.29 and π/2 in Q2.30 are the same integer, 0x6487ED51.

cordic_sincos.v

```
// ============================================================================
// cordic_sincos.v  (ENHANCEMENT: full-circle input)
// Wraps cordic_core with quadrant folding so the angle may be anywhere in
// -pi .. +pi.
//
// angle_in : Q3.29 radians (3 integer bits, 29 fraction bits), -pi .. +pi
// cos_out  : Q2.30
// sin_out  : Q2.30
//
// Folding (identities):
//   theta >  pi/2 : theta' =  pi - theta, cos = -cos(theta'), sin = sin(theta')
//   theta < -pi/2 : theta' = -pi - theta, cos = -cos(theta'), sin = sin(theta')
//   otherwise     : theta' = theta
// theta' is then shifted left by one bit (Q3.29 -> Q2.30) and sent to cordic_core.
//
// Latency is unchanged: 16 clocks after the start pulse.
// Inputs outside -pi .. +pi are not supported (wrap them first).
// ============================================================================
`timescale 1ns / 1ps
module cordic_sincos (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               start,
    input  wire signed [31:0] angle_in,   // Q3.29, -pi .. +pi
    output wire               done,
    output wire signed [31:0] cos_out,    // Q2.30
    output wire signed [31:0] sin_out     // Q2.30
);
    // pi and pi/2 in Q3.29 (pi/2 in Q2.30 has the same integer as pi in Q3.29)
    localparam signed [31:0] PI_Q329   = 32'h6487ED51;  // 3.14159265
    localparam signed [31:0] HALF_Q329 = 32'h3243F6A9;  // 1.57079633

    // ---- quadrant detection and folding (combinational) ----
    wire gt_half = (angle_in >  HALF_Q329);
    wire lt_half = (angle_in < -HALF_Q329);

    reg signed [31:0] folded;     // Q3.29, now within -pi/2 .. +pi/2
    always @(*) begin
        if (gt_half)
            folded = PI_Q329 - angle_in;
        else if (lt_half)
            folded = -PI_Q329 - angle_in;
        else
            folded = angle_in;
    end

    wire signed [31:0] angle_q230 = folded <<< 1;   // Q3.29 -> Q2.30

    // ---- remember whether cos must be negated for this run ----
    reg neg_cos;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            neg_cos <= 1'b0;
        else if (start)
            neg_cos <= gt_half | lt_half;
```

```
    end

    // ---- core ----
    wire signed [31:0] core_cos;
    cordic_core u_core (
        .clk      (clk),
        .rst_n    (rst_n),
        .start    (start),
        .angle_in (angle_q230),
        .done     (done),
        .cos_out  (core_cos),
        .sin_out  (sin_out)
    );

    assign cos_out = neg_cos ? -core_cos : core_cos;
endmodule
```

tb_cordic_sincos.v

```
// ============================================================================
// tb_cordic_sincos.v
// Self-checking testbench for cordic_sincos (full circle, Q3.29 input).
// ============================================================================
`timescale 1ns / 1ps
module tb_cordic_sincos;
    reg                clk;
    reg                rst_n;
    reg                start;
    reg  signed [31:0] angle_in;
    wire               done;
    wire signed [31:0] cos_out;
    wire signed [31:0] sin_out;

    cordic_sincos dut (
        .clk      (clk),
        .rst_n    (rst_n),
        .start    (start),
        .angle_in (angle_in),
        .done     (done),
        .cos_out  (cos_out),
        .sin_out  (sin_out)
    );

    initial clk = 1'b0;
    always #5 clk = ~clk;

    real TOL;
    integer errors;
    integer tests;

    function real q230_to_real;
        input signed [31:0] v;
        begin
            q230_to_real = $itor(v) / 1073741824.0;
        end
    endfunction

    // angle is Q3.29 here: scale by 2^29
    function signed [31:0] real_to_q329;
        input real r;
        begin
            real_to_q329 = $rtoi(r * 536870912.0);
        end
    endfunction

    function real absr;
        input real r;
        begin
            absr = (r < 0.0) ? -r : r;
        end
    endfunction

    task run_test;
        input real angle_rad;
        real exp_c, exp_s, got_c, got_s, err_c, err_s;
        integer cyc;
        begin
            tests = tests + 1;
            angle_in = real_to_q329(angle_rad);
            @(negedge clk);
            start = 1'b1;
            @(negedge clk);
            start = 1'b0;
            cyc = 0;
            while (!done && cyc < 100) begin
                @(negedge clk);
```

```
            cyc = cyc + 1;
        end
        exp_c = $cos(angle_rad);
        exp_s = $sin(angle_rad);
        got_c = q230_to_real(cos_out);
        got_s = q230_to_real(sin_out);
        err_c = absr(got_c - exp_c);
        err_s = absr(got_s - exp_s);
        $display("angle=%9.6f rad (%8.3f deg)  cycles=%0d",
                 angle_rad, angle_rad*57.29577951, cyc);
        $display("  cos: exp=%9.6f got=%9.6f err=%e", exp_c, got_c, err_c);
        $display("  sin: exp=%9.6f got=%9.6f err=%e", exp_s, got_s, err_s);
        if (!done || err_c > TOL || err_s > TOL) begin
            $display("  ** FAIL **");
            errors = errors + 1;
        end
        else
            $display("  PASS");
        repeat (2) @(negedge clk);
    end
endtask

integer k;
real    a;
initial begin
    $dumpfile("cordic_sincos_tb.vcd");
    $dumpvars(0, tb_cordic_sincos);
    TOL = 1.0e-4;
    errors = 0;
    tests  = 0;
    rst_n = 1'b0;
    start = 1'b0;
    angle_in = 32'sd0;
    repeat (4) @(negedge clk);
    rst_n = 1'b1;
    repeat (2) @(negedge clk);

    $display("==== cordic_sincos full-circle test (Q3.29 in, Q2.30 out) ====");

    // edge cases and each quadrant
    run_test( 3.1415926535);   // pi
    run_test(-3.1415926535);   // -pi
    run_test( 2.0);
    run_test(-2.5);
    run_test( 1.6);            // just past pi/2
    run_test(-1.6);
    run_test( 3.0);
    run_test( 1.5707963268);   // pi/2
    run_test(-1.5707963268);   // -pi/2

    // sweep -3.1 .. +3.1 rad in 0.1 steps
    for (k = -31; k <= 31; k = k + 1) begin
        a = k * 0.1;
        run_test(a);
    end

    $display("==== Done: %0d tests, %0d errors ====", tests, errors);
    if (errors == 0) $display("ALL TESTS PASSED");
    else             $display("SOME TESTS FAILED");
    $finish;
end
endmodule
```

5.5 Enhancement: Pipelined CORDIC (cordic_pipelined.v)

Give each of the 16 iterations its own hardware and a pipeline register. Throughput rises to one result per clock (latency 17 clocks). Area grows about 16 times for the adders and registers; the shifters become fixed wiring. tb_cordic_pipelined.v streams 31 angles back to back and checks that the results come out in order, one per clock.

cordic_pipelined.v

```
// ============================================================================
// cordic_pipelined.v  (OPTIONAL EXTENSION)
// Fully pipelined CORDIC, rotation mode, Q2.30, 16 stages.
// One new angle can be accepted every clock; result appears 17 clocks later.
// Same arithmetic as cordic_core.v, but each iteration has its own hardware.
// NOTE: provided as an extension starting point - simulate it with the
// testbench (drive valid_in / angle_in every clock) before relying on it.
// ============================================================================
```

```
`timescale 1ns / 1ps
module cordic_pipelined (
    input  wire               clk,
    input  wire               rst_n,
    input  wire               valid_in,
    input  wire signed [31:0] angle_in,   // Q2.30, -pi/2 .. +pi/2
    output wire               valid_out,
    output wire signed [31:0] cos_out,
    output wire signed [31:0] sin_out
);
    localparam signed [31:0] K_INIT = 32'h26DD3B6A;

    // arctangent constants as a function of stage index
    function signed [31:0] atan_c;
        input integer i;
        begin
            case (i)
                0 : atan_c = 32'h3243F6A9;
                1 : atan_c = 32'h1DAC6705;
                2 : atan_c = 32'h0FADBAFD;
                3 : atan_c = 32'h07F56EA7;
                4 : atan_c = 32'h03FEAB77;
                5 : atan_c = 32'h01FFD55C;
                6 : atan_c = 32'h00FFFAAB;
                7 : atan_c = 32'h007FFF55;
                8 : atan_c = 32'h003FFFEB;
                9 : atan_c = 32'h001FFFFD;
                10: atan_c = 32'h00100000;
                11: atan_c = 32'h00080000;
                12: atan_c = 32'h00040000;
                13: atan_c = 32'h00020000;
                14: atan_c = 32'h00010000;
                15: atan_c = 32'h00008000;
                default: atan_c = 32'h0;
            endcase
        end
    endfunction

    // pipeline registers: index 0 = input stage, index 16 = output
    reg signed [31:0] x [0:16];
    reg signed [31:0] y [0:16];
    reg signed [31:0] z [0:16];
    reg               v [0:16];

    // stage 0: load constants
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            x[0] <= 32'sd0; y[0] <= 32'sd0; z[0] <= 32'sd0; v[0] <= 1'b0;
        end else begin
            x[0] <= K_INIT; y[0] <= 32'sd0; z[0] <= angle_in; v[0] <= valid_in;
        end
    end

    // stages 1..16: one micro-rotation each
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : stage
            wire signed [31:0] xs = x[i] >>> i;
            wire signed [31:0] ys = y[i] >>> i;
            wire               zn = z[i][31];
            always @(posedge clk or negedge rst_n) begin
                if (!rst_n) begin
                    x[i+1] <= 32'sd0; y[i+1] <= 32'sd0;
                    z[i+1] <= 32'sd0; v[i+1] <= 1'b0;
                end else begin
                    x[i+1] <= zn ? (x[i] + ys) : (x[i] - ys);
                    y[i+1] <= zn ? (y[i] - xs) : (y[i] + xs);
                    z[i+1] <= zn ? (z[i] + atan_c(i)) : (z[i] - atan_c(i));
                    v[i+1] <= v[i];
                end
            end
        end
    endgenerate

    assign cos_out   = x[16];
    assign sin_out   = y[16];
    assign valid_out = v[16];
endmodule
```

tb_cordic_pipelined.v

```
// ============================================================================
// tb_cordic_pipelined.v
// Streams one new angle EVERY clock into cordic_pipelined and checks that the
// results come out in order, one per clock, after the 17-clock latency.
// ============================================================================
```

```
`timescale 1ns / 1ps
module tb_cordic_pipelined;
    localparam N = 31;                // number of angles streamed

    reg               clk;
    reg               rst_n;
    reg               valid_in;
    reg  signed [31:0] angle_in;
    wire              valid_out;
    wire signed [31:0] cos_out;
    wire signed [31:0] sin_out;

    cordic_pipelined dut (
        .clk       (clk),
        .rst_n     (rst_n),
        .valid_in  (valid_in),
        .angle_in  (angle_in),
        .valid_out (valid_out),
        .cos_out   (cos_out),
        .sin_out   (sin_out)
    );

    initial clk = 1'b0;
    always #5 clk = ~clk;

    real exp_c [0:N-1];
    real exp_s [0:N-1];
    real TOL;
    real a;
    real got_c, got_s, err_c, err_s;
    integer n_sent, n_got, errors;

    function real q230_to_real;
        input signed [31:0] v;
        begin
            q230_to_real = $itor(v) / 1073741824.0;
        end
    endfunction

    function signed [31:0] real_to_q230;
        input real r;
        begin
            real_to_q230 = $rtoi(r * 1073741824.0);
        end
    endfunction

    function real absr;
        input real r;
        begin
            absr = (r < 0.0) ? -r : r;
        end
    endfunction

    // checker: runs at every clock, results must arrive in the order sent
    always @(posedge clk) begin
        if (rst_n && valid_out) begin
            got_c = q230_to_real(cos_out);
            got_s = q230_to_real(sin_out);
            err_c = absr(got_c - exp_c[n_got]);
            err_s = absr(got_s - exp_s[n_got]);
            if (err_c > TOL || err_s > TOL) begin
                $display("FAIL #%0d: cos %f (exp %f)  sin %f (exp %f)",
                         n_got, got_c, exp_c[n_got], got_s, exp_s[n_got]);
                errors = errors + 1;
            end
            else
                $display("ok   #%0d: cos %f  sin %f", n_got, got_c, got_s);
            n_got = n_got + 1;
        end
    end

    initial begin
        $dumpfile("cordic_pipe_tb.vcd");
        $dumpvars(0, tb_cordic_pipelined);
        TOL = 1.0e-4;
        errors = 0;
        n_got = 0;
        rst_n = 1'b0;
        valid_in = 1'b0;
        angle_in = 32'sd0;
        repeat (4) @(negedge clk);
        rst_n = 1'b1;
        @(negedge clk);

        // one angle per clock, -1.5 .. +1.5 rad
        for (n_sent = 0; n_sent < N; n_sent = n_sent + 1) begin
            a = (n_sent - 15) * 0.1;
            exp_c[n_sent] = $cos(a);
            exp_s[n_sent] = $sin(a);
```

```
            angle_in = real_to_q230(a);
            valid_in = 1'b1;
            @(negedge clk);
        end
        valid_in = 1'b0;

        // drain the pipeline
        repeat (30) @(negedge clk);
        $display("==== results received: %0d of %0d, errors: %0d ====", n_got, N, errors);
        if (n_got == N && errors == 0) $display("ALL TESTS PASSED");
        else                           $display("SOME TESTS FAILED");
        $finish;
    end
endmodule
```

5.6 Physical Layout (3D View)

Figures 5 to 8 show the final physical layout of the CORDIC processor, first as a top view and then as tilted 3D views, in the normal colouring and in a highlighted (green) layer view. Reading the layout views:

- The standard cells sit in regular horizontal rows over the whole core, and evenly spaced vertical power stripes run from top to bottom.
- The logic and its wiring are concentrated in the upper part of the core; the input and output pin connections are visible along the top edge.
- The lower part of the core shows only the regular cell-row pattern with almost no routing, so the floorplan could be tightened to reduce the die area.

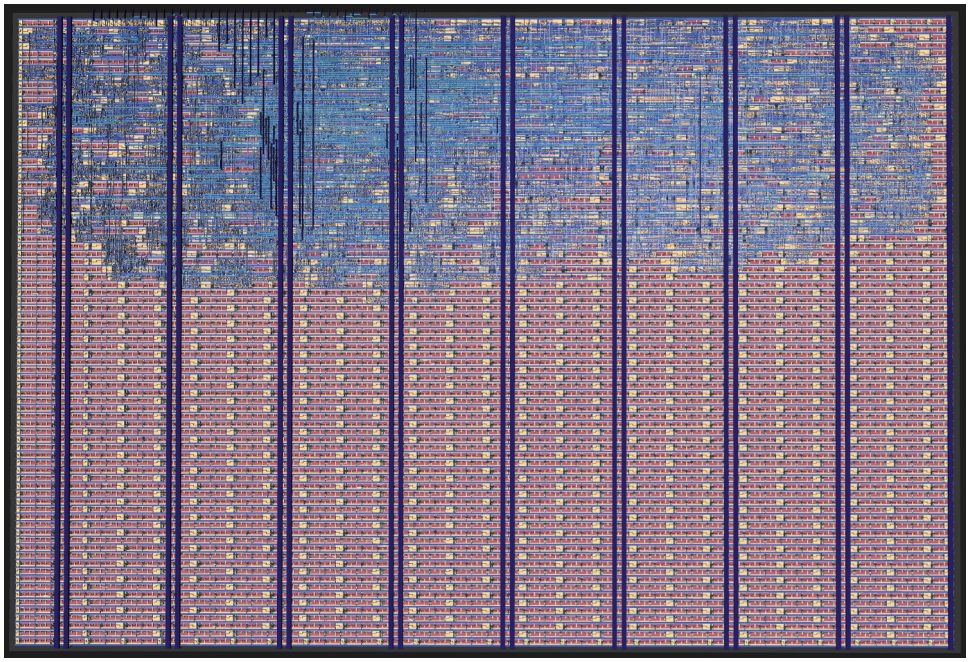


Figure 5: Top view of the placed and routed CORDIC processor layout.

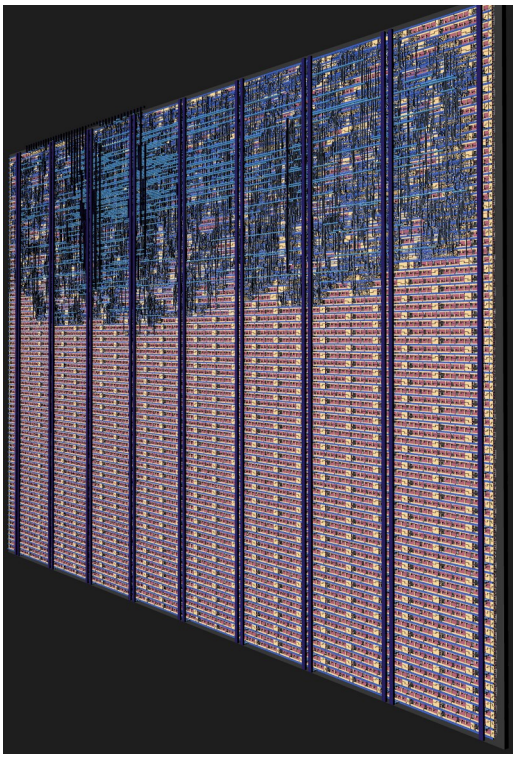


Figure 6: Tilted 3D view of the same layout, showing the cell rows and the vertical power stripes.

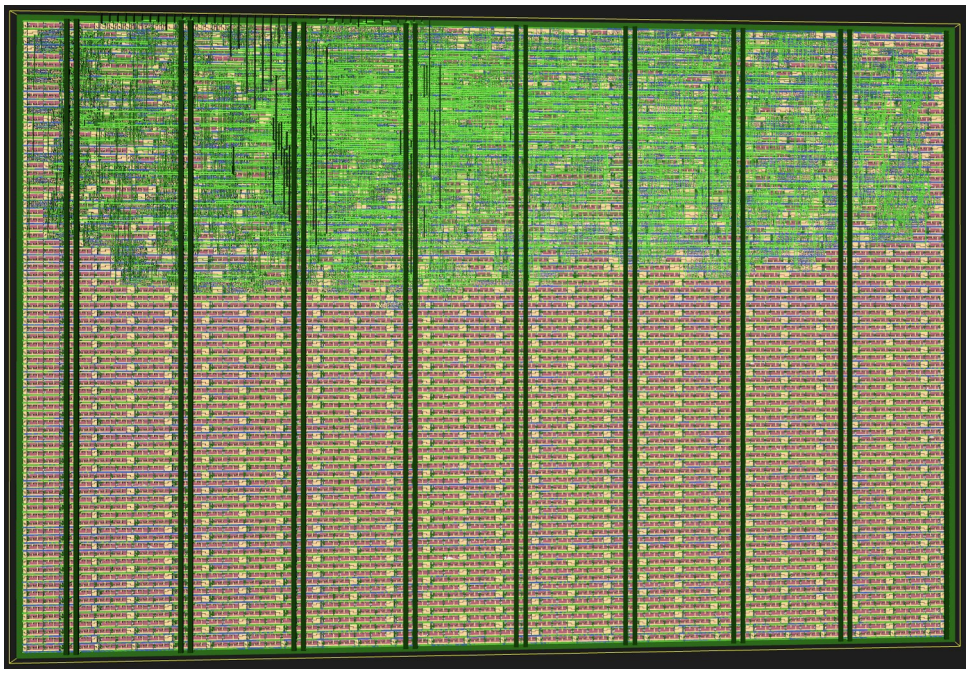


Figure 7: Top view in the highlighted (green) layer view; the wiring is concentrated in the upper part of the core.

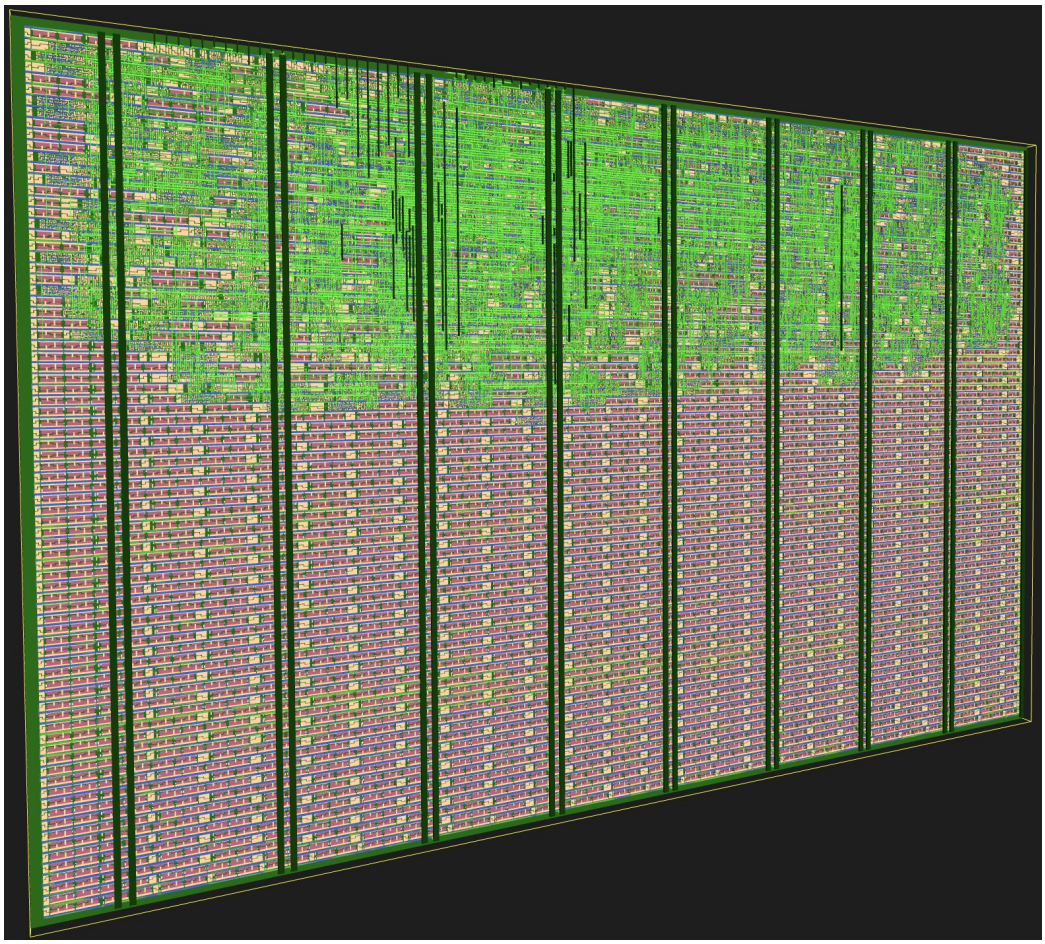


Figure 8: Tilted 3D view in the highlighted (green) layer view.

6. Testing, Results and Improvements

6.1 Test Plan and Results

Test / specification	Target value	Observed result	Outcome
Core testbench: 12 hand-picked angles (0°, ±30°, ±45°, ±60°, ±90°, 5.73°, −11.46°, 70.73°)	Error ≤ 1e-4	Worst error 1.8e-05	Pass
Full-circle testbench: 72 angles	Error ≤ 1e-4	Worst error 2.75e-05	Pass
Dense 0.5° sweep over −180° ... +180°	Error ≤ 1e-4	Worst error 3.01e-05	Pass
Latency	16 clock cycles	16 clock cycles (done after micro-rotation 15)	Pass

Working of testbench

- Generates a 100 MHz clock and a reset.
- For each angle: converts it to Q2.30, pulses start, waits for done, and compares X and Y with $cos and $sin.
- Tolerance is 1e-4. The CORDIC accuracy is about 3e-5, so a pass proves the design is working.
- Tests 12 hand-picked angles (0°, ±30°, ±45°, ±60°, ±90° and a few random values) and a sweep from −1.5 to +1.5 rad in 0.1 rad steps: 43 tests in total.
- Prints PASS/FAIL per test and ALL TESTS PASSED at the end, and writes cordic_tb.vcd.

- The last block prints X, Y and Z after every rotation (sampled on the falling clock edge, so the values are the registers after the rising edge). "rotations done = 0" is the freshly loaded K, 0, angle; "rotations done = 16" is the result.

Worked example: θ = 30°

Input conversion: θ = 30° = 0.5235987756 rad. Q2.30 value = 0.5235987756 × 2^{30} = 562209904 = 0x2182A470.

The table shows the register contents after each micro-rotation, as real numbers (value / 2^{30}). In the testbench console, row i appears as "rotations done = i+1". X and Y spiral in toward cos 30° = 0.866025 and sin 30° = 0.500000 while Z shrinks to zero.

Rotation i	Direction d	X (cos)	Y (sin)	Z (residual)
load	-	0.607253	0.000000	0.523599
0	+1	0.607253	0.607253	-0.261799
1	-1	0.910879	0.303626	0.201848
2	+1	0.834973	0.531346	-0.043130
3	-1	0.901391	0.426975	0.081225
4	+1	0.874705	0.483312	0.018806
5	+1	0.859602	0.510646	-0.012434
6	-1	0.867581	0.497215	0.003190
7	+1	0.863696	0.503993	-0.004623
8	-1	0.865665	0.500619	-0.000716
9	-1	0.866643	0.498928	0.001237
10	+1	0.866155	0.499775	0.000260
11	+1	0.865911	0.500198	-0.000228
12	-1	0.866033	0.499986	0.000016
13	+1	0.865972	0.500092	-0.000106
14	-1	0.866003	0.500039	-0.000045
15	-1	0.866018	0.500013	-0.000015

Final: X = 0.866018 (true 0.866025), Y = 0.500013 (true 0.500000). Error ≈ 1e-5.

Results for the testbench angles

Angle	cos exact	cos CORDIC	sin exact	sin CORDIC	max error
0.00°	1.000000	1.000000	0.000000	-0.000018	1.8e-05
30.00°	0.866025	0.866018	0.500000	0.500013	1.3e-05
45.00°	0.707107	0.707118	0.707107	0.707096	1.1e-05
60.00°	0.500000	0.500013	0.866025	0.866018	1.3e-05
90.00°	-0.000000	-0.000018	1.000000	1.000000	1.8e-05
-30.00°	0.866025	0.866018	-0.500000	-0.500013	1.3e-05
-45.00°	0.707107	0.707118	-0.707107	-0.707096	1.1e-05
-60.00°	0.500000	0.500013	-0.866025	-0.866018	1.3e-05
-90.00°	-0.000000	-0.000018	-1.000000	-1.000000	1.8e-05
5.73°	0.995004	0.995005	0.099833	0.099826	7.4e-06
-11.46°	0.980067	0.980065	-0.198669	-0.198675	6.1e-06
70.73°	0.329993	0.329993	0.943983	0.943983	2.5e-07

Expected hex outputs (X = cos, Y = sin)

Angle	X register (cos_out)	Y register (sin_out)
0.0°	0x3FFFFFFE	0xFFFFB636
30.0°	0x376CD741	0x200034EC
45.0°	0x2D416ADA	0x2D410EBF
60.0°	0x200034EC	0x376CD741
90.0°	0xFFFFB636	0x3FFFFFFE
-30.0°	0x376CD741	0xDFFFCB14
-45.0°	0x2D416ADA	0xD2BEF141
-60.0°	0x200034EC	0xC89328BF
-90.0°	0xFFFFB632	0xC0000002

Full-circle results (comparison with ideal values)

Angle (rad)	Angle (deg)	cos exact	cos CORDIC	sin exact	sin CORDIC	max error
3.1416	180.00°	-1.000000	-1.000000	0.000000	-0.000018	1.8e-05
-3.1416	-180.00°	-1.000000	-1.000000	-0.000000	-0.000018	1.8e-05
2.0000	114.59°	-0.416147	-0.416171	0.909297	0.909287	2.4e-05
-2.5000	-143.24°	-0.801144	-0.801160	-0.598472	-0.598450	2.2e-05
1.6000	91.67°	-0.029200	-0.029197	0.999574	0.999574	2.9e-06
-1.6000	-91.67°	-0.029200	-0.029197	-0.999574	-0.999574	2.9e-06
3.0000	171.89°	-0.989992	-0.989992	0.141120	0.141126	5.8e-06
1.5708	90.00°	-0.000000	-0.000018	1.000000	1.000000	1.8e-05
-1.5708	-90.00°	-0.000000	-0.000018	-1.000000	-1.000000	1.8e-05

All 72 test angles (the 9 above plus a −3.1 ... +3.1 rad sweep) have a worst error of 2.75e-05; a dense 0.5° sweep over −180° ... +180° gives 3.01e-05.

6.2 Problems and Improvements

Problem / limitation	Likely cause	Modification / future improvement
Input range limited to ±π/2	Q2.30 cannot represent π; 16 iterations reach 99.88°	Quadrant-folding wrapper cordic_sincos.v (Q3.29 input)
Error limited to about 3e-5	Residual angle atan(2^-15) after 16 iterations	More iterations with a longer arctangent table
One result every 16 clocks	Iterative architecture	Pipelined version: one result per clock, about 16 times the area
Pipelined version: simulation results not yet tabulated	tb_cordic_pipelined.v streams 31 angles but its output is not listed here	Run the testbench and tabulate the results
Lower part of the layout almost unused	Core area larger than the logic needs	Tighten the floorplan to reduce die area

6.3 Final Outcome

The processor meets its main objectives: it computes cosine and sine with shifts and adds only, in 16 clock cycles, with a worst error of 3.01e-05 against the 1e-4 testbench tolerance, and all test angles in the self-checking testbenches pass.

7. Estimated Cost

Item / component	Quantity	Cost per unit (₹)	Total cost (₹)
Hardware components (none purchased)	—	—	0
Software design work (Verilog simulation in Xilinx Vivado, Python model)	—	—	0
Fabrication / other expenses	—	—	0
Grand total			0

The cost is estimated. This is a software (RTL) design, and no physical components were purchased.

8. Conclusion and Future Scope

8.1 Conclusion

A multiplier-free CORDIC processor was designed in Verilog and simulated in Xilinx Vivado. Using Q2.30 fixed-point arithmetic and 16 shift-and-add micro-rotations, it returns cosine and sine of an input angle in 16 clock cycles with an absolute error of about 3e-5 (roughly 15 bits). The design uses a synchronous datapath controlled by a simple counter, is fully synthesizable, and can be extended with a pipelined architecture, full-circle input and further functions such as tangent and vector magnitude. The final physical layout of the processor is presented as top and 3D views in Section 5.6.

8.2 Future Scope

- Add further functions such as tangent and vector magnitude.
- Increase the number of iterations, with a longer arctangent table, for higher accuracy.
- Tighten the floorplan and report synthesis utilization and timing results to complete the physical-design evidence.

9. Team Contribution

Student name / roll number	Main contribution
Arsh Satsangi / 2400941 (sole team member)	Complete project work: study of the CORDIC algorithm, Verilog design, testbenches, Python reference model, physical layout views and report writing

References

[1] J. E. Volder, “The CORDIC trigonometric computing technique,” IRE Transactions on Electronic Computers, vol. EC-8, no. 3, pp. 330–334, Sept. 1959.

[2] J. S. Walther, “A unified algorithm for elementary functions,” in Proc. AFIPS Spring Joint Computer Conference, 1971, pp. 379–385.

[3] R. Andraka, “A survey of CORDIC algorithms for FPGA based computers,” in Proc. ACM/SIGDA Int. Symp. on Field Programmable Gate Arrays (FPGA ’98), 1998, pp. 191–200.

Appendix

Appendix A. Python reference model

cordic_model.py

```
"""Bit-accurate Python model of cordic_core.v and cordic_sincos.v (16 iterations).

cordic_core  : angle in Q2.30, valid range -pi/2 .. +pi/2
cordic_sincos: angle in Q3.29, valid range -pi .. +pi (folds into the core's range)
Run:  python3 cordic_model.py
"""
import math, json

FRAC = 30
SC = 1 << FRAC
LUT = [0x3243F6A9, 0x1DAC6705, 0x0FADBAFD, 0x07F56EA7, 0x03FEAB77, 0x01FFD55C, 0x00FFFAAB, 0x007FFF55,
       0x003FFFEB, 0x001FFFFD, 0x00100000, 0x00080000, 0x00040000, 0x00020000, 0x00010000, 0x00008000]
K_INIT = 0x26DD3B6A
PI_Q329 = 0x6487ED51      # pi in Q3.29 (same integer as pi/2 in Q2.30)
HALF_Q329 = 0x3243F6A9    # pi/2 in Q3.29

def s32(v):
    v &= 0xFFFFFFFF
    return v - (1 << 32) if v & 0x80000000 else v

def to_q(r, frac=FRAC):   # same as Verilog $rtoi(r * 2^frac): truncate toward zero
    return s32(int(r * (1 << frac)))

def to_r(v, frac=FRAC):
    return v / (1 << frac)

def cordic_q(z, trace=False):
    """Run the 16 micro-rotations on a Q2.30 angle word z. Returns (x, y, z, trace)."""
    x, y = K_INIT, 0
    tr = []
    for i in range(16):
        xs, ys = x >> i, y >> i          # Python >> on negative ints is arithmetic
        if z < 0:
            x, y, z = s32(x + ys), s32(y - xs), s32(z + LUT[i])
        else:
            x, y, z = s32(x - ys), s32(y + xs), s32(z - LUT[i])
        if trace:
            tr.append((i, to_r(x), to_r(y), to_r(z)))
    return x, y, z, tr

def cordic(angle_rad, trace=False):
    return cordic_q(to_q(angle_rad), trace)

def sincos_full(angle_rad):
    """Model of cordic_sincos: Q3.29 input, quadrant fold, core, conditional negate of cos."""
    a = to_q(angle_rad, 29)
    gt, lt = a > HALF_Q329, a < -HALF_Q329
    folded = PI_Q329 - a if gt else (-PI_Q329 - a if lt else a)
    x, y, _, _ = cordic_q(s32(folded << 1))
    return (s32(-x) if (gt or lt) else x), y

# angles exactly as written in tb_cordic_core.v
TB_ANGLES = [0.0, 0.5235987756, 0.7853981634, 1.0471975512, 1.5707963268,
             -0.5235987756, -0.7853981634, -1.0471975512, -1.5707963268, 0.1, -0.2, 1.2345]
TB_ANGLES += [k * 0.1 for k in range(-15, 16)]
# angles exactly as written in tb_cordic_sincos.v
FULL_ANGLES = [3.1415926535, -3.1415926535, 2.0, -2.5, 1.6, -1.6, 3.0, 1.5707963268, -1.5707963268]
FULL_ANGLES += [k * 0.1 for k in range(-31, 32)]

if __name__ == "__main__":
    for i, v in enumerate(LUT):
        assert v == round(math.atan(2 ** -i) * SC), i
    K = 1.0
    for i in range(16):
        K /= math.sqrt(1 + 2 ** (-2 * i))
    assert K_INIT == round(K * SC)
    assert PI_Q329 == round(math.pi * 2 ** 29) and HALF_Q329 == round(math.pi / 2 * 2 ** 29)
    print("LUT, K_INIT and Q3.29 constants verified")

    rows, worst = [], 0.0
    for a in TB_ANGLES:
        x, y, z, _ = cordic(a)
        ec, es = abs(to_r(x) - math.cos(a)), abs(to_r(y) - math.sin(a))
        worst = max(worst, ec, es)
        rows.append(dict(angle=a, deg=math.degrees(a), cos_hw=to_r(x), sin_hw=to_r(y), cos_ref=math.cos(a),
                         sin_ref=math.sin(a), ec=ec, es=es, xhex="%08X" % (x & 0xFFFFFFFF), yhex="%08X" % (y &
0xFFFFFFFF)))
    print("core testbench angles:", len(TB_ANGLES), "  worst abs error: %.3e" % worst)

    dense = 0.0
    for k in range(-180, 181):
        a = math.radians(k * 0.5)
        x, y, _, _ = cordic(a)
        dense = max(dense, abs(to_r(x) - math.cos(a)), abs(to_r(y) - math.sin(a)))
    print("dense -90..+90 sweep (361 pts): %.3e" % dense)

    frows, fworst = [], 0.0
    for a in FULL_ANGLES:
        x, y = sincos_full(a)
        ec, es = abs(to_r(x) - math.cos(a)), abs(to_r(y) - math.sin(a))
```

```
        fworst = max(fworst, ec, es)
        frows.append(dict(angle=a, deg=math.degrees(a), cos_hw=to_r(x), sin_hw=to_r(y), cos_ref=math.cos(a),
                          sin_ref=math.sin(a), ec=ec, es=es, xhex="%08X" % (x & 0xFFFFFFFF), yhex="%08X" % (y &
0xFFFFFFFF)))
    print("full-circle testbench angles:", len(FULL_ANGLES), "  worst abs error: %.3e" % fworst)

    fdense = 0.0
    for k in range(-360, 361):
        a = math.radians(k * 0.5)
        x, y = sincos_full(a)
        fdense = max(fdense, abs(to_r(x) - math.cos(a)), abs(to_r(y) - math.sin(a)))
    print("dense -180..+180 sweep (721 pts): %.3e" % fdense)

    _, _, _, tr = cordic(0.5235987756, trace=True)
    json.dump(dict(rows=rows, worst=worst, dense=dense, trace=tr, frows=frows, fworst=fworst, fdense=fdense,
                   n_core=len(TB_ANGLES), n_full=len(FULL_ANGLES)),
              open("/home/claude/cordic/results.json", "w"))
```

Appendix B. File list

File	Purpose
cordic_atan_lut.v	Arctangent table, Q2.30
cordic_core.v	Top module: datapath, step counter, done flag
tb_cordic_core.v	Self-checking testbench with convergence printout
cordic_sincos.v	Enhancement: full-circle wrapper (Q3.29 input, quadrant folding)
tb_cordic_sincos.v	Testbench for the wrapper
cordic_pipelined.v	Enhancement: pipelined version
tb_cordic_pipelined.v	Streaming testbench for the pipelined version
cordic_model.py	Bit-accurate Python reference model (core and wrapper)
CORDIC_Processor_Project_Guide_v3.pdf	This guide

— End of Report —